S P E C T R A V I D E O

U S E R S G R O U P -- Wgtn

Volume 3, Number 1 -- July 1986

The next meeting is on Monday, July 21st at 7.30 in the Staff Common Room , Wellington Clinical School , Mein Street Newtown. The next will be on the 18th of August, and the following on the 15th of October 1986. (ie as usual the 3rd Monday of the month.)

Wellington SV Users Group

Committee Members

Chairman ----- Darryl Alison Ph 85522 (Paraparaumu)

Secretary ----- Joseph Dol 889275

Treasurer ----- Ross Fletcher 788454

Don Stanley 896379

Garry Clark 367872

Newsletter Committee
Steve Hanson 843495
John Matthews 327805

Other specific areas of responsibility for committee members are
Don -- Software Consultant
Garry -- Radio Access & public domain tape software;
John -- Library;
Darryl -- Program evaluation committee chairman & Kapiti users representative.

MORE FROM ACTION COMPUTERS

Don has advised that 80 column cards may be ordered through Action Computers - as mentioned at the last meeting. A deposit of $150 - to Don by 22 July will confirm your order.

GAMES SCORES

Spectron	12910	Robert Campin
Sector Alpha	2445200	Garry Clark
Kung Fu	82435	Garry Clark
	48705	Garry Clark (*)
S C Force	748600	Deirdre Chin
Telebunny	113520	Garry Clark
	32720	Garry Clark (+)
	31670	Kalima Collinson (*)
Omac	35360	Richard Bremford
	25700	Don Stanley (+)
Ninja	251350	Nicola Tuinman
Tetra	284450	Julian Stone
Turboat	68390	Garth Thornton (*)
Turboat	104610	Taane Clark
Sasa	172950	Steve Lacey
Frantic Fred	21720	Rachel Hughes (level4)
Flipper Slip	37640	Garth Thornton
Bats Alive	3540	Colin Chin (*)
Galaxy	1447	"
Jumping Jack	2600	"
TV Nightmare	7040	"
	3640	" (*)
	4980	Deirdre Chin (+)

(+) = started in normal level
(*) - started in arcade level
otherwise practice level or unknown.

Although it is nice not to have to update the scores all the time, it would be nice to see a few more changes in the scores above. C'MON all you gamesmasters - show us how good your skills are !!

Club Sponsor

Action Computers -- P.O Box 7442, Wellington South.

Photocopying from Library

Anybody requiring articles from the library to be photocopied please send request to the LIBRARIAN, not any other committee member. We have had problems with articles not being copied when requested, please ensure that requests go to the librarian.

Also membership renewals should be sent to Box 26050, Newlands, Wellington, not to any other box, otherwise it is possible that the mailing list will not get updated.

AND NOW FROM THE "YOU BLEW IT ..
(it had to happen sometime)
... JOHN !!" DEPARTMENT

Last month's Pascal Graphics demo program had a few unfortunate bugs which, on the whole, stopped it from working. As I have recieved some new, quicker routines since then, here's the whole program again:

```
program lines;
var x1,x2,y1,y2,colour,spare:byte;
    key:char;

procedure screen1;
begin
  inline($f3/
         $3e/$01/
         $32/$3a/$fe/
         $21/$10/$36/
         $cd/$51/$e6/
         $fb);
end;

procedure Cls;
begin
    inline($21/$77/$37/
           $cd/$51/$e6);
end;

procedure Line( tlx,tly,brx,bry,colour,style:byte);
var location:integer;
begin
  case style of
   0 : location:=$243c;
   1 : location:=$2452;
   2 : location:=$2401;
   else exit;
   end;
  if not (colour in [0..15]) then exit;
  mem[$fe3a]:=1;
  mem[$fa13]:=colour;
  mem[$f343]:=0;
  mem[$fe47]:=0;
  mem[$fe42]:=brx;
  mem[$fe46]:=brx;
  mem[$fe45]:=0;
  mem[$fe49]:=0;
  mem[$fe44]:=bry;
  mem[$fe48]:=bry;
  inline($06/$00/$3a/tlx/$4f/$16/$00/$3a/tly/$5f/
         $3e/$00/$32/$2b/$fe/$2a/location/$cd/$51/$e6);
end;
```

Cont.....

```
begin
  mem[$fe3a]:=1;
  mem[$Fe3b]:=3;
  screen1;
  Cls;
  repeat
    x1:=random(235);
    x2:=x1+random(254-x1);
    y1:=random(171);
    y2:=y1+random(190-y1);
    colour:=random(16);
    spare:=random(3);
    line(x1,y1,x2,y2,colour,spare);
  until keypressed;
  read(kbd,key);
end.
```

The main error in last months program was the call to the ROM routine, which was supposed to clear the graphics screen. I had been trying to call the Screen command in ROM, and put the address of that routine in instead of the CLS call. Both of these calls are shown correctly in the above program.

-----+-----

A note to would-be contributers

Don't let *Anyone* or *Anything* put you off. The newsletter is made from your contributions, if we don't receive any, we can only produce a poor magazine.

Presently we are lacking in BASIC programs and in hints to beginners, and from beginners. We try to cater to ***all*** club members, but lately our we have been receiving mainly specialty items, which are of little benefit to many. Among the many things we would like to see arriving in the mail (addressed to *The Editors*) are:

- Programs - Basic, Pascal, Assembly, YouNameIt.
- Programming hints, and techniques.
- Utilities - any type, as long as they are self explanatory or come with instructions.
- News - anything you think other club members might like to know.
- Problems - we will find someone to answer them if we can't.

Everything **we receive will be considered.**

Notes From The Last Meeting.

As members who attended last month's meeting are aware, an area of concern discussed at the last committee meeting by the committee was raised for discussion and the proposed remedy put to the members for approval.

The item of concern was the sudden loss of our printer, who was giving the club an extremely good rate (far below the market rate) for printing the newsletters, and going to the situation of having to have the newsletter printed commercially. The resultant price increase was approximately 200%, ie going from around $80 to between $200 to $220 per run of 200 magazines. At that rate, the club was heading for a serious cash flow problem; and together with the other major expense, postage, was actually incurring a loss with every issue.

The Solutions.

We have been able to locate a printer who has quoted a very reasonable price (in comparison to the above examples) for the printing of next issue. After that an ongoing quote will be given which will only be subject to adjustment for increases in materials. Even though he has given a good price, it is still almost double the $80 we were being charged previously.

Therefore the following motions were proposed by Steve Hanson and voted on:

1 THAT the annual subscription for New Zealand residents be raised to $18.00 effective from 16 June 1986. Seconded by Joseph Dol.

CARRIED unanimously

2 THAT the format of the magazine be changed to one magazine issued two-monthly. This is to cut down on postage. Seconded by Don Stanley.

CARRIED unanimously

3 THAT the subscriptions for individual overseas members be increased to NZ$30 for Australasian residents and to NZ$40 for residents anywhere else,
AND
The price for bulk purchase by Users groups be increased to $2.50 per issue,
AND
The price for back issues, single copy purchase by individuals, etc, be increased to $3.00 each. Seconded by Garry Clark.

CARRIED unanimously.

While price increases are regretted, we feel that even at $18 per year this represents extremely good value for money. I'm sure that many members will put there assent to this.

Don Stanley informed the meeting that Tom Johnson of South Auckland Computers is going to import SVI/MSX to NZ himself. The retail price of the 738 has been increased to $1365. SAC are having a sale which includes SVI software tapes $5, Software Disks $10, MSX adapters, 64k RAM cards $69, 16k RAM cards $50, MSX cartridges $69.95. Write to South Auckland Computers, PO Box 720, PAPAKURA, Sth Auckland for further details.

Joseph Dol

Public Domain CP/M

Anyone wanting anything from the library of CP/M PD software, please write to the Librarian at the club address, specifying either which disc, or which programs you would like. As my drives are single sided, a whole disc from the list now occupies up to 2 full discs. Most of the programs on the discs are *'Squeezed'*, this means they have been run through a function in New Sweep (NSW, Nsweep etc. but not the earlier Sweep) so as to take up as little room as possible. They all have a 'Q' as the 2nd letter in their extension, and can be easily unsqueezed by using New Sweep to tag them and then the 'Q' option to unsqueeze them, usually onto another disc. Please send discs, rather than asking us to purchase them for you, and as my drives are slightly *"Non-Standard"*, you would be unable to read them if formatted on my drives, so please send them formatted. Please also include, either a stamped return mailer, or add 60c to your payment to cover postage. We will return your disk in your packaging.

VIEWING WORDSTAR FILES

It is common to use the NSWEEP program to "View" or print out Wordstar files. I accidentally found that there is another way: using PIP. Try

```
    PIP CON:=A:(textfilename)[Z]      (To view a Wordstar File)
or  PIP LPT:=A:(textfilename)[Z]      (To print out a W.S. File)
```

The [Z] parameter sets the parity bit (eighth bit) of all ASCII bytes to zero during the copy operation.
Partial copies can be made using PIP's options; for example use Q(string)^Z (or S(string)^Z) to copy up to (or from) the named string in the file.

Jim Oliver

Editor's *(John's)* Note:

This may be useful with a self written Pascal or SVMBasic program, for example a help file, that you want to write using a word-processor, but don't want to have to get rid of the garbage it produces when printed yourself.

THESE CHANGING TIMES

I am sorry to tell you folks - but there is little that we can do about it. The cost of producing our bulletin has been doubled for our last four issues and the club can no longer absorb the costs.

If you had been at our last meeting (in June), you will already be aware of the problems which have caused the increases in subscription rates. For those of you who were unable to attend, I will outline the basic points for your information.
Firstly, there is the problem of the increased cost per issue for production. Our previous annual subscription of $15 covered a total of eleven issues, at a basic cost of just over 80 cents per issue.
The difference between the basic newsletter cost and the subscription rate went to cover a whole host of club services:
Meeting Venue Costs
Media Costs for storing the Public Domain programs (copying is done voluntarily)
Magazine subscriptions (or in the case of other Users Group Newsletters, a newsletter exchange is made ie cost of newsletter and {Overseas} postage) to maintain our Library
The remainder was used by the club to cover the (inevitable) cost increases, and to provide a 'Buffer' fund.
This buffer had in the past been used to cover the initial cost of such activities as - the club's BASIC manual, produced in bulk then sold to members at cost, or for some form of Social function as we have had in the past.
The increase in bulletin costs to now approx $1.50 per issue has almost wiped the buffer fund. To maintain the bulletin in its current format, while still protecting the club from cost increases would have meant a subscription increase to around $20.00 or more.

This brings us to the next decision - that of changing the format to a bi-monthly issue.
Neither John, nor myself have sufficient time to produce an issue on our own. We concede that Don was in his time able to do so, however we have found that it really is a two-man job.
By changing the frequency of our newsletter, we can maintain our standards, while reducing the pressure on us to find time to meet, and compile each issue.
Finally, - on parting I wish to add that we have been supported well by some members supplying articles, but the reduced frequency of the newsletter does not mean a reduced need for good articles.
PLEASE keep them coming so that we can produce bumper editions, which will appeal to **all levels of SV user** ... after all, John and I are only 'Editors' ... what you get out of the newsletter is dependent upon what you are prepared to put in.

Steve.

BACK TO BASICS

The following article is one of a series published in the Spectravideo Australasian Users Group Newsletter in early 1984. We are publishing this in response to many requests to get "Back to BASICs (excuse the pun! - Ed)". Many new users have come to the group since the recent discounted sales of SV hardware, and also a number of people buying second hand gear. So - Old Hands please bear with us and remember - you needed to know the basics once yourself.

EXPLORING BASIC - PART 1

BY L. A. Dunning

All BASIC programs are stored in memory. A self evident point you may say? Perhaps, but consider the ramifications - firstly only RAM can be used so that the program can be changed, secondly there is only a limited amount of RAM available, so every byte used is valuable. The first point is necessary if you want to do anything other than play the same video game. The second point forces any designer of BASIC languages to create a system of representing adn interpreting the program using a minimum of memory.

MICROSOFT realised this and have opted for a tried and tested system, as used on the TRS80 and other computers. This is the TOKENISATION of BASIC programs in memory. What this means, is that every key word or statement used by teh BASIC interpretor, is represented by a single or double Byte in memory. If you compare for example the 7 Bytes needed to represent 'RESTORE' in ASCII as opposed to the 1 Byte of 8C(Hexadecimal, ie Base 16).

However, things are not quite that simple since a method is needed to differentiate betweent these Tokens, and string values or numeric values. This is one reason why program lines are used in BASIC programs, they provide a simple "box" for program statements. The format for all BASIC lines in memory is:

```
    0/LN/HN/L#/H#/Statements . . . . . /0
      / NL  / N#  /
```

Where 0 represents a zero byte; LN & HN are the LOW and HIGH bytes of NL, the next line pointer which indicates where in memory the next line is; L# & H# are the LOW and HIGH bytes of N#, the Line Number; Statements.. represent the coded form of the program line; and the 0 at teh end is the end of Line pointer. When one program line follows another in memory, the starting and ending zeroes are combined. Using two bytes to indicate the next line in memory means that successive lines do not have to follow one another in memory. Lines can be edited without disturbing existing lines by placing the result in a free area and adjusting pointers.

The start of any program has two leading 0 bytes and the end of a program is noted by having two zeroes for the next line pointer.

LISTING 1 can be used to demonstrate this. It uses teh VARPTR statement to locate a literal string in memory and show peeks of the surrounding memory. Use it now before further reading. This shows the starting address of the line, NL pointer, N# and gives a dump of the bytes (In hexadecimal and ASCII) that makes up the line.

How does listing one work? It works because variables such as numbers and names must also be represented in memory. BASIC maintains pointers for each variable used or created, so that it can be manipulated by the program. This is an overhead and requires a variable number of bytes dependent upon variable type. The statement VARPTR gives a result that points to the first byte of each pointer. Assuming K is the value given by the statement VARPTR(A), then PEEKing the following memory locations will give the following results:-

```
A=INTEGER    A=Single Precision    A=Double Precision
---------    ------------------    ------------------
 K - LSB      K - LSB               K - LSB
K+1- MSB     K+1- NEXT MSB         K+1- NEXT MSB
             K+2- MSB              K+2- NEXT MSB
             K+3- EXPONENT VALUE   K+3- NEXT MSB
A$=STRING                          K+4- NEXT MSB
---------                          K+5- NEXT MSB
 K -LENGTH OF STRING               K+6- MSB
K+1- LSB OF STRING LOCATION        K+7- EXPONENT VALUE
K+2- MSB OF STRING LOCATION
```

LSB means Lowest Significant Byte and MSB means Most Significant Byte in the values represented. For strings, PEEK(K) is the same as LEN(A$). Using the formula X = PEEK(K+1)+PEEK(K+2)*256, X will give us the location of the string in Memory. This brings us to the next subject: LITERAL VARIABLES.

A Literal Variable is one that can be read directly in a listing of the program, such as A$="FRED", or PI=3.1415926 . These are read when the program first passes through that program line and the location is in that line. When the variable is changed for any reason, however BASIC allocates new space for the result and a new location is found in a variable work-area. Provided the string variable is unchanged, it can be used as a pointer to the location of the line in memory. On other computers that have the VARPTR Statement (such as the TRS80) it is used to put otherwise unobtainable characters into literal strings, to put Machinecode routines on lines themselves and provide a method of allowing equations for INPUT statements. Listing 2 demonstrates the first of these.

To use listing two properly, you must type GOTO 10 to set up the Function Keys, Edit the string on Line 160, then type GOTO 110 to convert the string. The 6-9 Keys will be used to indicate twhere cursor control characters are to be placed, and the 5 key is used to place CHR$(27) in the string. Keys 1-4 show arrow symbols. Try it and see the result. The only limitation is that once changed

by this method, the line cannot be changed normally without destroying the values poked therein. Converting a string twice brings it back to the original condition.

Other strings can be converted by changing the variable in the VARPTR statement, provided the line the variable is on has been read first. You should renumber, or alter the program to suit yourself. You may have noticed that some of the pokes produced strange results when you listed the program, this is due to the routines used in the listing - and I will be discussing this in part two.

LISTING 1.

```
  20 Rem Show how lines are composed.
  30 CLS
  40 DEF FNH$(A)=STRING$(2-LEN(HEX$(A)),"0")+HEX$(A):
     DEF FNL(A,B)=A+256*B
  50 GOSUB 170:K=VARPTR(A$):X=FNL(PEEK(K+1),PEEK(K+2))-8
  60 PRINT" Addr.  N.Line Number"
  70 FOR L = 0 TO 4: S=X
  80 PRINTUSING" #####  #####  ##### ";S;FNL(PEEK(S),PEEK(S+1));
     FNL(PEEK(S+2),PEEK(S+3))
  90 S=X+4
 100 P=PEEK(S):PRINT" ";FNH$(P);:S=S+1
 110 IF P()0 GOTO 110 ELSE PRINT
 120 S=X+4
 130 P=PEEK(S):S=S+1:IF P)31 AND P(123 THEN PRINT "  ";CHR$(P);
     ELSE PRINT"    ";
 140 IF P() 0 GOTO 130 ELSE X=S:PRINT
 150 NEXT
 160 LIST 170-
 170 A$="":J=9
 180 REM A TEST
 190 '
 200 RETURN
```

LISTING 2

```
  10 REM  Listing 2a -- Sets up Characters
  30 SCREEN 0: RESTORE 60
  40 FOR A = 1 TO 9:KEY A ,CHR$(211+A):NEXT
  50 FOR A = 4000 TO 4031: VPOKE A+40,VPEEK(A) XOR 255:NEXT
  60 FOR A = 4032 TO 4039: READ B: VPOKE  A,B:NEXT:
     DATA 0,0,48,48,48,48,0,0
  70 STOP
  80 :
  90 REM  listing 2b -- Converts String
 110 GOSUB 160:K=VARPTR(A$):X=PEEK(K+1)+PEEK(K+2)*256
 120 J=PEEK(X):IF J=34 OR J=0 GOTO 150
 130 IF J(32 THEN POKE X,J+189 ELSE IF J)215 THEN POKE X,J-189
 140 X=X+1:GOTO 120
 150 CLS:LOCATE 10,10:PRINT A$:LOCATE 0,20:STOP
 160 A$="This is a test\ZZZZZZZZZZZZZZXpand so is this!
     Xq\\ZZZZZZZZZZZZZand this"
 170 RETURN
```

FURTHER WORDSTAR PATCHES

Anyone out there running the Custom Format CP/M that's been around a while now, - the one that provides more Disk space, ability to read/write several Disk Formats, plus display function keys as in BASIC - may have come across several problems when running their old software. Here are a few that I have found, and how to solve them.

Firstly, anyone running Wordstar will have found that the cursor keys don't work any more! This is because Steve McNamee used the same method Digital Research used in the standard SV CP/M. They used the ROM routines to read the keyboard. Steve does not, however use DR's trick of mapping the cursor keys to the CP/M standard, ie ^X,^E,^S,^D. Instead the ASCII codes 1C to 1F (HEX) are returned.

The solution: whenever a control key is pressed in Wordstar, it is checked with a table of possible commands, and a jump occurs if a match is found. This table starts at 0489H (use DDT for this as in previous issues) and works like this -

Two bytes which are expected character codes
Two more bytes holding the jump address

So to solve the problem (in DDT) type

s04B1 (RETURN)

This should show 18(HEX) as the current value - type

1F (RETURN)

to change it. The next byte should be zero - (press return to check) if it isn't change it. Leave Edit mode (. (RETURN) at an empty line) then follow the same steps to patch:

04B5 from 05 to 1E
049D from 13 to 1D
04A5 from 04 to 1C

the second byte in each case should be zero.

While you're there - there is another change that you can make! If you would rather have DEL delete the character under the cursor (as ^G does) and (Backspace) delete the previous character, patch the four bytes beginning at 04A1 to read:

08 00 9A 64

and at 0541 to read:

7F 00 64 68

This solves your problems with Wordstar - if anyone knows similar fixes for DBASE II and other programs - let us know!!

Turbo Pascal programs compiled under the normal CP/M won't work unless they are recompiled under the custom CP/M format, as they expect there to be the same amount of memory available as there was at compile time. The custom BIOS uses more memory than the normal one and so there may not be sufficient memory for a program to run under the custom format even if it had been running happily under the original format.

HACKERS CORNER

DON STANLEY

VARIABLE TABLE

When your program is typed in BASIC keeps track of where it finishes by using a series of 'pointers' . The start of each line of code is the address in memory where the next line starts. You can look at these by PEEKing the first few bytes.

```
FOR I=&H8001 TO &h800F:?HEX$(I)"  "HEX$(PEEK(I)):NEXT
```

which will list the first 15 bytes of the program in hex. Note the 8001. This is where all BASIC programs start unless you change an address in high memory (BASIC's work area). We won't get into that here, because it causes all sorts of problems unless you really know the system well.

Eventually BASIC will find a pointer which points to an address of 0000. This is the signal that the program is finished.

Now type in ? HEX$(256 * PEEK(&HF7EF) + PEEK(&HF7EE)) . This value is the start of the variable table, which is where all the variables in the program get their values stored. It always occurs 2 bytes after the end of the program.

KEYBOARD BUFFER

When you type a character on the keyboard it goes into the keyboard buffer. This is an area of memory 40 bytes long which the system constantly scans for a RETURN (ENTER). When a return is found, all the characters up to the return are interpreted by BASIC, and the command is carried out.

This buffer is pointed to by addresses FA1A/1B and FA1C/1D. The first 2 are the position in the buffer where the next character you type in will go, the last 2 are the position where BASIC last read an instruction from. For instance typing in RUN will leave the last address unchanged, but the first address will advance by 3. Then pressing ENTER tells BASIC to read all those characters (RUN) and work out what to do with them. At this stage the 2 addresses will be the same.

The buffer can be used by machine code programmers to access ROM routines. Your machine code program simply inserts the instruction into the buffer, ensuring that the address pointed to by FA1C/1D does not get updated. Insert a return in as well, don't forget to update the first address, and then return control to BASIC.

HOOKS

Throughout the roms there are hooks. These are calls from the

roms into an area of memory known as the hook table. These calls generally do nothing except go straight back to where the hook was called from. (The disk system is accessed by hooks).

We can utilise the hooks (provided we know where in rom they occur) by altering the area in ram that they call. Usually this consists of a JUMP to somewhere else in ram where a machine code program is run.

Knowing where the hooks occur requires a disassembled listing of the machine code roms. All calls which call a location between FE79 and FFB1 are calls to hooks. The technical documentation which tells where rom routines are is available from the Wellington Users Group, although it is by no means complete.

MAIN

This is a rom routine where BASIC spends much of its time. In SVI it begins at 09C4H. MAIN is an area of code which BASIC constantly loops through looking for a RETURN (ENTER) typed at the keyboard. All BASIC instructions (direct or program) are initiated from MAIN.

MAIN begins with a hook. This hook calls location FE94, which is generally a machine code return unless you overwrite it. One way in which a program can overwrite it is to have locations FE94/95/95 jump to a machine code routine somewhere in ram. Why would you want to do this? One reason concerns program protection and will be explained in detail in Bits & Bytes in August and September. Briefly you alter the hook to jump to a machine code program which zeroes out the first line pointer at 8001H. Basic thinks the program does not exist.

On its own thats not much use, loading the program still can't use the hook unless it is run. But what if the BASIC program is BSAVE'd. The idea is to zero out the 8001/02 bytes, BSAVE the memory image, thus forcing the user to RUN the program when it is BLOAD'ed back in. The address to RUN the program from at BLOAD time would be the address of a small machine code program which loaded the bytes at 8001/02 and the variable table address at F7EE/EF back in. The machine code routine puts RUN and a RETURN in the keyboard buffer, then jumps to MAIN+3 (the +3 is to avoid the hook). The program is RUN, returning control to MAIN at completion. MAIN calls the hook, which jumps to another small machine routine which blots the program back out (zero out 8001/02 , or really kill it by zeroing from 8001 to the start of the variable table!), then resets the hook to just a machine code return. No program can be listed easily, and the fact that it is saved as memory image makes copying more difficult. And you can adapt this as much as you like, use more than just 8001/02 or something, if you are really experienced with machine code you should be able to lead any hacker a merry chase by putting a few obscure machine code instructions in.

This is all explained in far more detail including software to make it a bit easier in aforementioned issues of Bits & Bytes.

MSX PROGRAM REVIEWS

by Paul Ridgway.

A review follows of 3 MSX programs supplied by Action Computers, Box 7442, Wellington. All programs are priced at $24.95 on tape, $27.95 on 5.25" disk and $34.95 on 3.5" disk.

DUNGEON DELVERS DELIGHT - ZIRAK'S GEM

If exploring dungeons for treasure and slaying monsters is your idea of fun, then you'll love Zirak's Gem. This game is not one to fill in an idle half an hour. If you plan to complete it, you will need to set aside a nice long evening or rainy afternoon. The game is written in basic, so it is no 'Ghostbusters' or 'Buck Rodgers'. However, it is most enjoyable.

The aim of the game is to recover a magic gem which will save your village of Turamber from destruction. The gem is located somewhere in the depths of a dungeon complex, which is home for an assortment of monsters.

Naturally you start at level one. While you can move freely up and down between levels, you cannot hope to beat the monsters at the lower levels until you have built up enough skill and experience points (by thrashing the weaker nasties) at the higher levels.

As a sideline, there are pieces of gold to be collected at each level which can be redeemed at the local temple for experience points. The amounts of gold you find increase in size the further down into the dungeon you go. To add some spice to the game there are are also some 'mystery' items. Until you land on these, you do not know whether they are a beneficial spell to aid your quest, or a trap which will land you further in it.

What you see on the screen is a little man representing you. As you move him around, his torch lights up the level and the floorplan is revealed. When you change levels a status table (showing your abilities, experience and spells) is shown while the next level is being randomly generated. Thus no two games will be the same.

The documentation contains all you need to know and consists of six pages which explain the background, terminology, spells, traps symbols and how to move around. The MSX version has an additional page, and requires at least 32K of RAM and a joystick to run.

The verdict from my tribe of software demolishers (aged 7-12): "good", "real neat", "choice" and "I like it". Personally I found it a most addictive game to play and it even dragged my wife to the keyboard, which is no mean feat.

MATHEMATICS WHIZ - GRAPH PLOTTER

Do you remember all those graphs you had to draw at school to do with SIN and TAN and all that stuff? Well, it would certainly have been nice to have had Graph Plotter available then.

Graph Plotter is a general purpose program for plotting a y value as a function of x which is selected by the user. The documentation provided consists of 7 pages of quite detailed instructions on using the various options, but it is assumed that you know your maths and that you know what you are doing. A full example of each option would have been useful in demonstrating how they operated (for those of us who are a bit rusty in the maths department), otherwise they appear clear and comprehensive.

The program presents you with a main menu of eight choices that can be selected with function keys (F8 returns you to Basic). Most of the options require that you enter a function (via F7), such as x*sin(x),x+1/x,etc. before the option is selected.

To plot a graph (F1) several parameters need to be entered first. These are to set the high and low points on the Y-axis, and the start and end points on the X-axis. It is also possible to alter the area that the graph will occupy and to specify whether a discrete or continuous graph is wanted. The SEL key will produce a screen print to a printer - most of the common printers are catered for.

There are two methods to solve equations - the SECANT method (F2) and the Newton Raphson Method (F3). The secant method requires no knowledge of the derivative which has to be supplied for the Newton Raphson method. Both require you to enter the number of iterations, and as the equation is being solved the intermediate results are printed on the screen.

Numerical integration of any 2 dimensional function can be done with Simpsons method (F4). The lower and upper limits are requested and when resolved the integral value is displayed, along with an estimate of the error in the result.

Option F5 provides a method of numerical differentiation, which supplies an approximation based on the interpolating polynomial. The GAUSS-SEIDEL method (F6) is used to solve linear systems with up to 10 unknowns and an invertible coefficient matrix (according to the instructions).

This is a serious program, and as such, is not one that would suit everybody. If you are doing a course of mathematics at high school or university, or if you deal with mathematical functions with some frequency, then you will undoubtedly find that this program will be an invaluable aid.

MSX DISASSEMBLER - 'DISASS'

This is a well presented and easy to use disassembler for the 64K MSX computer. A disassembler is normally used to translate machine code into human readable assembler mnemonics. This program goes beyond this and provides several additional features as well.

DISASS, written by Don Stanley, has eight options which are controlled by the function keys. F1 provides a toggle switch to direct the output to the screen or the printer. F8 and CTRL/STOP return you to Basic or the main menu at any time.

Three dumps are provided which ask you to supply a start and end address in hex. The output is printed in groups of 8 characters in ASCII format (F2) or in Hex (F7).

Z80 mnemonics are produced when the range is selected with F3. When I used this option to print out the 32K ROM on my machine, it took nearly 3 hours on my Star SG10 - so be warned.

F4 prompts for a start address and then allows you to keep poking bytes into memory from there. You can load machine code programs which may have been hand assembled, between D400 up to E500 (on a cassette system). You can then use DISASS to check that your codes are correct.

The ability to search for a string of up to 15 bytes (F5) or an ASCII string (F6) are both provided. These can be quite fun and useful to play around with, to locate the start addresses of all sorts of things.

The documentation consists of a single page of instructions which clarify and expand on the use of the function keys. This is all that is required.

All these features make this an extremely useful tool for the person that wishes to delve into the workings of their computer. For those wishing to program in machine code it is invaluable. There is still about 10K of RAM available after DISASS is loaded and running. If you are even slightly interested in what's going on in your MSX - then this is the program for you.

The following listing is a Mastermind-style game. The player must guess a two-digit number. The game will tell you if you have the correct digits, and if the digits are in the correct location.

This program courtesy of Jim Megenis.

→

```
100 REM Number guessing game:"Patatika"
110 REM by Jim Megenis, Coromandel
120 REM Spectravideo 328 Version
130 ON STRIG GOSUB 500
140 GOSUB 500
150 CLS:T=0:J$="try"
160 N$=RIGHT$(STR$(TIME),2)
170 A=VAL(LEFT$(N$,1))
180 B=VAL(RIGHT$(N$,1))
190 PRINT"What number do you guess (00 - 99)";
200 INPUT Z$
210 IF LEN(Z$)<> 2 THEN PRINT"bad entry - go again":GOTO190
220 IF ASC(LEFT$(Z$,1))>57 OR ASC(LEFT$(Z$,1))<48 THEN PRINT"bad entry ; go agai
n":GOTO190
230 IF ASC(RIGHT$(Z$,1))>57 OR ASC(RIGHT$(Z$,1))<48 THEN PRINT"bad entry , go ag
ain":GOTO190
240 T=T+1
250 X=VAL(LEFT$(Z$,1))
260 Y=VAL(RIGHT$(Z$,1))
270 IF X=A AND Y=B THEN 350
280 CLS:PRINT"You guessed ";X;Y;" This scores ";
290 IF X=A OR Y=B THEN PRINT"1 TIKA !":GOTO330
300 IF X=B AND Y=A THEN PRINT"2 PATA !":GOTO330
310 IF X=B OR Y=A THEN PRINT"1 PATA !":GOTO330
320 PRINT"zero"
330 PRINT:PRINT"You have had ";T;J$
340 J$="tries":GOTO190
350 CLS:LOCATE12,5:PRINT"TWO TIKA !"
360 LOCATE5,10:PRINT"Hurrah; You guessed it !"
370 PRINTTAB(5)"The number was ";A;B
380 PRINTTAB(5)"you did it in ";T;J$
390 PRINT:PRINT:PRINT:PRINTTAB(3)"Do you want to play again ?";
400 A$=INKEY$:IF A$=""THEN 400
410 IFA$="y"ORA$="Y" THEN 150 ELSEPRINT:PRINT"game ended":END
500 STRIG(0) OFF:CLS:PRINT"  **    How to play Patatika    **"
510 PRINT:PRINT
520 PRINT"The computer chooses a number between"
530 PRINT"00 and 99.You try to guess this number"
540 PRINT:PRINT"Type two digits, then press Enter."
550 PRINT"If either digit is in the number"
560 PRINT"you will score 'one pata'."
570 PRINT"If both your digits are in the number"
580 PRINT"you will score 'two pata'."
590 PRINT"If one of your digits is both in the"
600 PRINT"number, and in its right place, then"
610 PRINT"you will score 'one tika'"
620 PRINT"If both digits are in the right place"
630 PRINT"you score 'two tika', that is to say"
640 PRINT"          you have DONE IT!"
650 PRINT:PRINT
660 PRINT"Anytime: Space + Enter gives this page"
670 PRINT:PRINT"Press Enter when ready to play    ";
680 INPUT; G$
690 T=T-1:CLS:STRIG(0) ON
700 RETURN
```